The SECD Machine

A Tutorial Introduction

October 24, 2026

Abstract

The SECD machine, introduced by Peter Landin in 1964, was the first abstract machine for evaluating expressions of the λ-calculus, and it remains the standard first example of how a functional language can be implemented. This tutorial builds the machine up from three ideas: replacing substitution by *environments*, representing functions as *closures*, and making the evaluator's own call stack explicit as the *dump*. We give the transition rules, trace several programs step by step, state what the machine computes, and program it with Church encodings up to a recursive factorial. We then look inside a small C implementation, including a pointer-sharing bug that is instructive in its own right, and show how the machine turns into a compiler target. A section on related machines and a set of exercises close the paper. The only prerequisite is a working knowledge of the untyped λ-calculus.

Contents

1 Introduction **2**

2 What we assume **3**

3 From substitution to a machine **3**
3.1 Environments instead of substitution . . . 3
3.2 Closures: functions carry their environment . . . 3
3.3 Control and stack: evaluation as postfix code . . . 4
3.4 The dump: an explicit call stack . . . 5

4 The machine **5**
4.1 States . . . 5
4.2 Transitions . . . 5
4.3 The machine as a loop . . . 6

5 Worked examples **7**
5.1 The identity function . . . 7
5.2 Two arguments: $K\ x\ y$ . . . 8
5.3 Lexical scope in action . . . 9

6 What the machine computes **9**
6.1 Values and read-back . . . 9
6.2 Consequences . . . 10

7 Programming the machine **10**
7.1 Booleans and numbers . . . 10
7.2 Obstacle 1: a strict *if* . . . 10
7.3 Obstacle 2: recursion without Y . . . 11

7.4 Factorial . . . 11
7.5 What it costs . . . 11

8 Primitives and constants **11**

9 Inside an implementation **12**
9.1 Representation . . . 12
9.2 From rules to code . . . 13
9.3 A lesson in sharing . . . 13
9.4 Memory . . . 13

10 Compiling for the SECD machine **14**
10.1 Instructions . . . 14
10.2 The compiler . . . 14
10.3 Conditionals . . . 15
10.4 Tail calls . . . 15

11 Relatives and descendants **16**

12 Summary **17**

Exercises **17**

A Reference card **18**

1 Introduction

A λ-term on paper is evaluated by *rewriting*: find a redex $(\lambda x.M)\,N$, replace it by $M[N/x]$, and repeat. This is a fine mathematical definition, but it is a poor description of what a computer does. Each step has to search the term for the next redex, copy N into every occurrence of x, and rename bound variables to avoid capture. None of this resembles running a program, where code is fixed, data lives in memory, and a stack records what to do after a function returns.

An *abstract machine* closes that gap. It is a state-transition system whose states are made of simple data structures (stacks, lists, tables) and whose transitions each do a small, bounded amount of work. It sits between the mathematical semantics, which says *what* a term means, and a real implementation, which says *how* the bits move.

Landin's SECD machine [11] was the first such machine for the λ-calculus. Its name lists its four registers:

Stack	values computed so far, waiting to be used;
Environment	the values of the variables currently in scope;
Control	the expressions still to be evaluated;
Dump	saved copies of S, E and C, to resume after a function returns.

By the end of this tutorial you should be able to run the machine by hand, explain why each register is needed, read and modify an implementation, and recognise the SECD machine's ideas in the implementations of real languages.

Companion code. The examples are executed by a short C program, `secd23.c`, which follows the presentation of Field and Harrison [7]. All traces, step counts and the plot in section 7 were produced by running it. Section 9 walks through the program.

2 What we assume

We use the untyped λ-calculus with the usual conventions: application associates to the left, so $M\ N\ P$ means $(M\ N)\ P$, and the body of a λ extends as far right as possible.

$$M, N ::= x \ \mid\ \lambda x.M \ \mid\ M\ N$$

A term may contain free variables. We treat these as names of *constants* supplied from outside, for example $x = 4$ and $y = 3$. This lets the examples produce readable results without first adding numbers to the language.

Call-by-value. The SECD machine implements *call-by-value* evaluation. The *values* are λ-abstractions and constants. An application $M\ N$ is evaluated by evaluating both M and N to values and then, if the function is $\lambda x.B$ and the argument is V, continuing with $B[V/x]$. Two consequences matter throughout:

1. Arguments are evaluated *before* the call, even if the function never uses them. If the argument diverges, the whole call diverges.
2. Evaluation stops at a λ. We never evaluate inside the body of a function that has not been called. The term $\lambda b.(\lambda z.z)\,4$ is already a value. A term in this state, with no redex outside any λ, is said to be in *weak head normal form* (WHNF)[1].

A running example. Let $K = \lambda a.\lambda b.a$. With $x = 4$ and $y = 3$, call-by-value evaluation by substitution goes

$$K\,x\,y \ = \ (\lambda a.\lambda b.a)\,4\,3 \ \to \ (\lambda b.4)\,3 \ \to \ 4. \tag{1}$$

We will run the same computation on the machine in section 5 and see each of these two β-steps reappear in a different form.

3 From substitution to a machine

The machine is built from three ideas. Each one fixes a specific problem with evaluation by rewriting.

3.1 Environments instead of substitution

Substitution $B[V/x]$ copies V into the body B, and so creates a new term at every call. The alternative is to leave B untouched and remember, in a separate table, that x now stands for V. When evaluation later meets x, it looks the value up. This table is the *environment*.

We write an environment as a list of bindings, most recent first, such as $E_0 = [x \mapsto 4,\ y \mapsto 3]$. Extending an environment adds a binding at the front:

$$E_0[a \mapsto 4] = [a \mapsto 4,\ x \mapsto 4,\ y \mapsto 3].$$

Lookup searches from the front, so a newer binding of a name *shadows* an older one. Because extension only adds a new cell in front of an existing list, the old list is shared rather than copied. Environments created by different calls therefore form a tree of linked cells (figs. 1 and 8).

3.2 Closures: functions carry their environment

With environments a new problem appears. Suppose a λ-term is evaluated, passed somewhere else, and called later. Its body may refer to variables that were in scope where the λ was *written*. By the time it is called, the environment in force may have changed. Consider

$$T \ = \ (\lambda f.\,(\lambda x.\,f\,x)\ y)\ (\lambda z.\,x) \qquad \text{in } E_0 = [x \mapsto 4,\ y \mapsto 3]. \tag{2}$$

[1] Strictly, the C program's tag name `WHNF` covers all the machine's values. For call-by-value these are exactly the closures and constants.

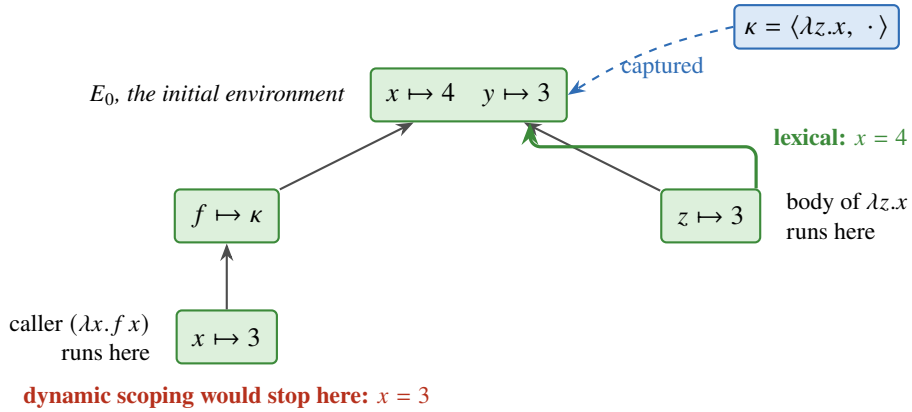


Figure 1: Environments while evaluating T from (2). Arrows point from each environment to the older one it extends. The closure κ for $\lambda z.x$ captured E_0. When κ is called with 3, its body runs in $[z \mapsto 3]$ extending E_0 (right branch), so looking up x finds 4. The caller's environment (left branch) is never consulted.

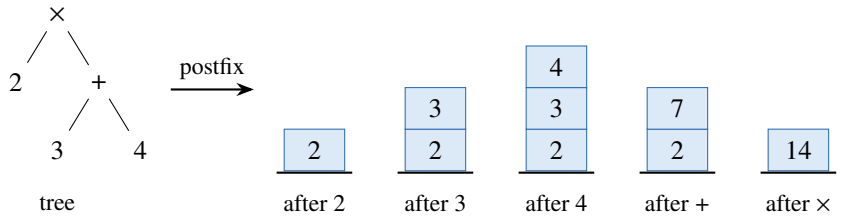


Figure 2: A stack calculator evaluating $2 \times (3 + 4)$. The tree is flattened into postfix order. Constants are pushed, and operators replace their operands by the result. The SECD machine does the same with application in place of + and ×, and the marker @ playing the role of the operator.

The function $\lambda z.x$ is created where $x = 4$. It is called inside $\lambda x.f\,x$, where x has been rebound to $y = 3$. Substitution gives

$$T \to (\lambda x.\,(\lambda z.4)\,x)\ 3 \to (\lambda z.4)\ 3 \to 4,$$

so the x in $\lambda z.x$ must mean the x that was visible *where the function was written*. This rule is called *lexical* (or *static*) scoping. To get it without substitution, a function value must remember the environment it was created in. The pair

$$\langle \lambda x.B,\ E \rangle$$

of a λ-term and an environment is called a *closure*. When the closure is called with argument v, its body B runs in $E[x \mapsto v]$: the closure's *own* environment extended with the parameter, rather than the caller's environment.

Using the caller's environment instead is called *dynamic* scoping, and for T it would give 3. Early Lisp systems behaved that way, which is known as the *funarg problem*. Landin's closures were the fix. Figure 1 shows the environments involved in evaluating T.

3.3 Control and stack: evaluation as postfix code

How does the machine decide what to evaluate next? Think of a pocket calculator evaluating $2 \times (3 + 4)$. The expression is a tree, but a stack calculator receives it in *postfix* order, 2 3 4 + ×: numbers are pushed onto a stack, and an operator pops its operands and pushes the result (fig. 2).

The SECD machine works the same way. The *control* C is the list of things still to be done, and the *stack* S holds the values produced so far. Function application plays the role of the operator. To evaluate $M\,N$, the machine replaces it on the control by the sequence

$$N,\ M,\ @$$

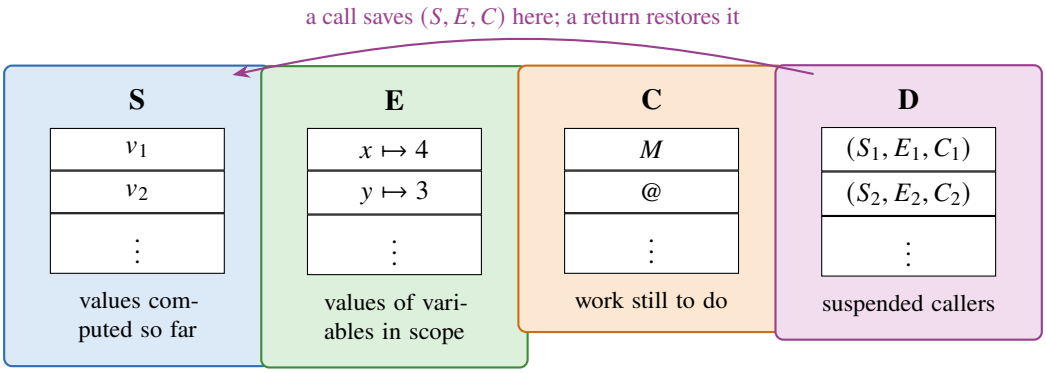


Figure 3: The four registers of the SECD machine. Each is a list, drawn with its head at the top. The dump holds complete copies of the other three registers, one per pending call.

That is: evaluate the argument, evaluate the function, then apply. The marker @ ("apply") is the machine's only operator symbol, and Landin wrote it *ap*. When @ reaches the front of the control, the function and its argument are the top two values on the stack.

Unlike the calculator, the machine flattens the tree *lazily*, one application node at a time, as each node reaches the front of the control. Section 10 shows that flattening the whole tree in advance turns the machine into a compiler target.

3.4 The dump: an explicit call stack

The last ingredient deals with function calls. When a closure $\langle \lambda x.B,\ E' \rangle$ is applied, its body B must run with a fresh stack, in the environment $E'[x \mapsto v]$, with B as the control. Afterwards the machine has to return to exactly where it was: the old stack (minus the function and argument), the old environment, and the rest of the old control.

The *dump* D saves that triple (S, E, C). A call pushes a triple onto the dump, and when the body's control runs out the machine pops it back. The dump is the machine's call stack, and each triple is an activation record. In modern terms it represents the *continuation*, everything that remains to be done once the current call finishes [4].

A recursive evaluator, written as an ordinary program, gets this bookkeeping for free from the call stack of the language it is written in. The point of the dump is to make that hidden stack visible as data. Every transition is then a constant amount of work, with nothing hidden in the implementation language.

4 The machine

4.1 States

Definition 1 (SECD state)**.** A *state* is a quadruple $(S,\ E,\ C,\ D)$ where

- S is a list of values (top first). A value is a constant, a primitive function, or a closure $\langle \lambda x.B,\ E \rangle$;
- E is an environment, a list of bindings $x \mapsto v$;
- C is a list of control items. Each is either a λ-term or the marker @;
- D is a list of saved triples (S', E', C').

We write $v : S$ for the list with head v and tail S, $[\,]$ for the empty list, and $[v_1, \ldots, v_n]$ for short lists. Figure 3 pictures a state.

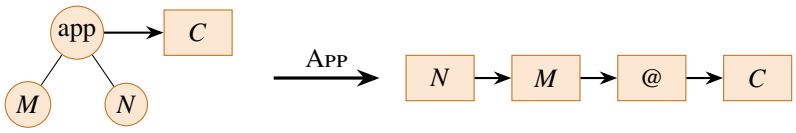
4.2 Transitions

The transition rules are given in fig. 4. Each rule looks only at the head of the control (or, when the control is empty, at the dump). Exactly one rule applies to any state that is not final or stuck. We name the

Var (1a)	$(S,\ E,\ x : C,\ D)$	$\longrightarrow (E(x) : S,\ E,\ C,\ D)$
Abs (1b)	$(S,\ E,\ \lambda x.B : C,\ D)$	$\longrightarrow (\langle \lambda x.B,\ E \rangle : S,\ E,\ C,\ D)$
App (1c)	$(S,\ E,\ (M\ N) : C,\ D)$	$\longrightarrow (S,\ E,\ N : M : @ : C,\ D)$
Call (1$d\,ii$)	$(\langle \lambda x.B,\ E' \rangle : v : S,\ E,\ @ : C,\ D)$	$\longrightarrow ([\,],\ E'[x \mapsto v],\ [B],\ (S, E, C) : D)$
Prim (1$d\,i$)	$(f : v : S,\ E,\ @ : C,\ D)$	$\longrightarrow (\delta(f, v) : S,\ E,\ C,\ D)$
Return (2)	$(v : S,\ E,\ [\,],\ (S', E', C') : D)$	$\longrightarrow (v : S',\ E',\ C',\ D)$

Start: $([\,],\ E_0,\ [M],\ [\,])$ evaluates M with free variables bound by E_0. **Halt:** $(v : S,\ E,\ [\,],\ [\,])$; the result is v.
Stuck: Var with x unbound, or @ with a constant in function position. Prim applies when f is a primitive; δ is its meaning (section 8).

Figure 4: The transition rules of the SECD machine.



control before: one tree, then C control after: argument, function, apply

Figure 5: App replaces one application node by three control items. This is one step of the postfix flattening from fig. 2.

rules, and the C program labels them with the case numbers in parentheses.

In words:

Var
A variable is evaluated by looking it up. Its value goes on the stack.

Abs
A λ is already a value, but it must remember the current environment, so the machine pushes a closure. The body is *not* evaluated. This is where the machine stops at WHNF.

App
An application is split into its parts, with the marker @ scheduled after both (fig. 5). Note the order: the argument N is evaluated *first*, so that when @ comes up the function is on top of the stack and the argument just below it. This right-to-left order is the one used by Landin and by the C program. For pure terms that terminate, the order does not change the result.

Call
The function is a closure. The machine saves the current stack, environment and control on the dump, then starts the body in the closure's environment extended with the argument (fig. 6, top).

Return
The control is empty, so the current body has finished with its result on top of the stack. The machine pops the saved triple and pushes the result onto the restored stack (fig. 6, bottom).

4.3 The machine as a loop

Figure 7 shows the transition rules as the control flow of an interpreter's main loop, which is how the C program is organised. Each pass through the loop performs exactly one transition. The loop ends when

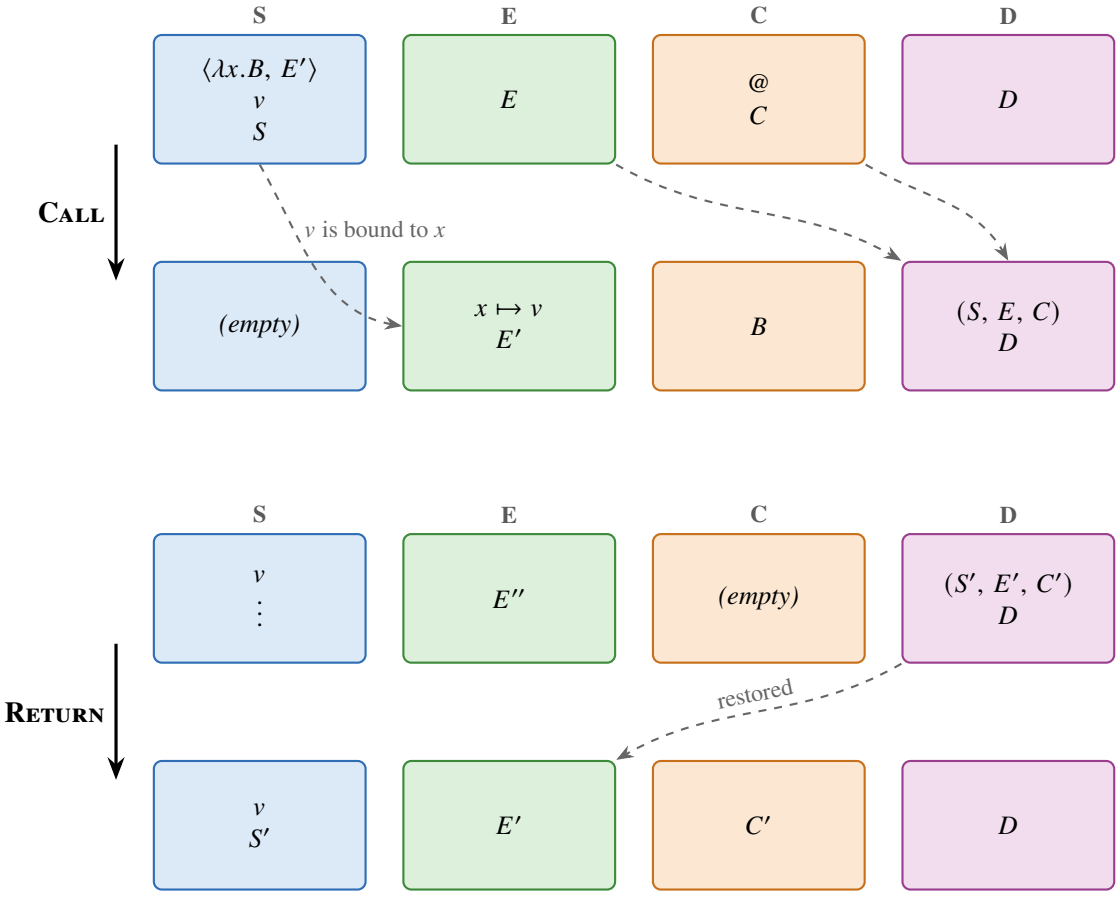


Figure 6: Calling and returning. **Top:** CALL consumes the closure and its argument, saves the rest of the state on the dump, and starts the body B on an empty stack in the closure's environment. **Bottom:** when the body's control is used up, RETURN restores the saved state and pushes the result v onto it. Note that the closure's environment E', not the caller's E, is what the body sees.

the control and the dump are both empty.

5 Worked examples

Throughout, $E_0 = [x \mapsto 4,\ y \mapsto 3]$. In the traces, each row shows the state *after* the rule named in the second column. Dump frames are written (S, E, C).

5.1 The identity function

Let $I = \lambda x.x$ and evaluate $I\ y$:

#	rule	S	E	C	D
0	start	$[\,]$	E_0	$[I\ y]$	$[\,]$
1	APP	$[\,]$	E_0	$[y,\ I,\ @]$	$[\,]$
2	VAR	$[3]$	E_0	$[I,\ @]$	$[\,]$
3	ABS	$[\langle I, E_0\rangle,\ 3]$	E_0	$[@]$	$[\,]$
4	CALL	$[\,]$	$[x \mapsto 3] \cdot E_0$	$[x]$	$[([\,], E_0, [\,])]$
5	VAR	$[3]$	$[x \mapsto 3] \cdot E_0$	$[\,]$	$[([\,], E_0, [\,])]$
6	RETURN	$[3]$	E_0	$[\,]$	$[\,]$

Here $[x \mapsto 3] \cdot E_0$ is E_0 extended with $x \mapsto 3$. In step 4 the new binding of x shadows the outer $x \mapsto 4$, so step 5 finds 3. The machine halts after step 6 with result 3. Even the identity function costs a

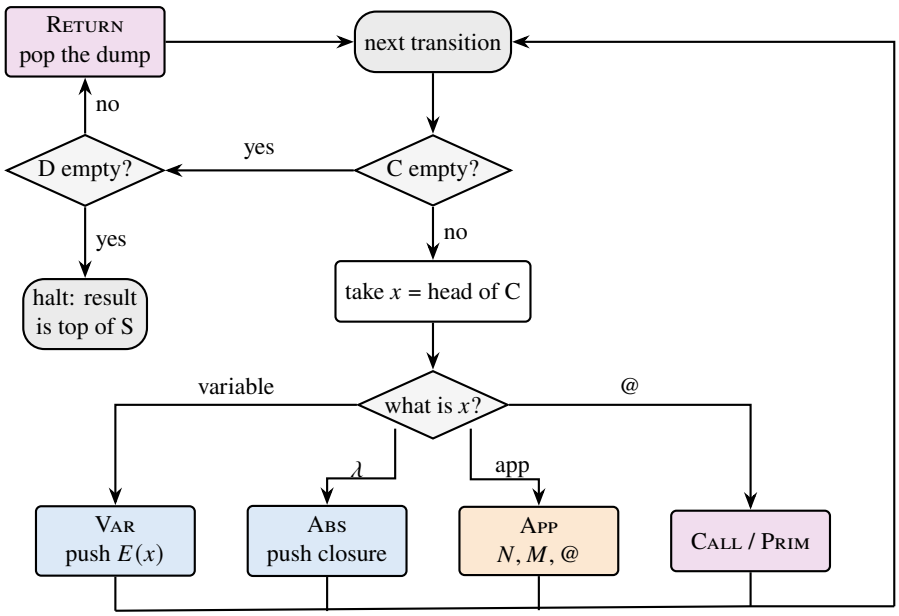


Figure 7: The machine as an interpreter loop. Every transition does a bounded amount of work, apart from the environment lookup in Var, which searches a list.

dump frame: the call in step 4 saves a frame even though nothing remains to be done after the call returns. Section 10 returns to this point under the heading of tail calls.

5.2 Two arguments: $K\ x\ y$

Now the running example $K\,x\,y$ with $K = \lambda a.\lambda b.a$. Recall that this means $(K\,x)\,y$. We abbreviate $E_1 = [a \mapsto 4] \cdot E_0$, $E_2 = [b \mapsto 3] \cdot E_1$, $\kappa_1 = \langle K,\ E_0 \rangle$ and $\kappa_2 = \langle \lambda b.a,\ E_1 \rangle$.

#	rule	S	E	C	D
0	start	[ ]	E_0	$[K\,x\,y]$	[ ]
1	App	[ ]	E_0	$[y,\ K\,x,\ @]$	[ ]
2	Var	[3]	E_0	$[K\,x,\ @]$	[ ]
3	App	[3]	E_0	$[x,\ K,\ @,\ @]$	[ ]
4	Var	[4, 3]	E_0	$[K,\ @,\ @]$	[ ]
5	Abs	$[\kappa_1,\ 4,\ 3]$	E_0	$[@,\ @]$	[ ]
6	Call	[ ]	E_1	$[\lambda b.a]$	$[([3], E_0, [@])]$
7	Abs	$[\kappa_2]$	E_1	[ ]	$[([3], E_0, [@])]$
8	Return	$[\kappa_2,\ 3]$	E_0	$[@]$	[ ]
9	Call	[ ]	E_2	$[a]$	$[([\,], E_0, [\,])]$
10	Var	[4]	E_2	[ ]	$[([\,], E_0, [\,])]$
11	Return	[4]	E_0	[ ]	[ ]

A few things to notice:

- **The two @ markers.** Step 3 splits the inner application and puts a second @ in front of the first. The inner call is made first (step 6). The outer @ is saved on the dump with the stack [3] and comes back into play only after the inner call returns (step 8).
- **A function returned as a result.** The inner call returns a closure, κ_2, rather than a number. Its environment E_1 records that $a = 4$. When the frame from step 6 is popped in step 8, the binding $a \mapsto 4$ disappears from the machine's E register but survives inside κ_2. This is exactly why environments cannot be freed when a call returns, as they would be on a conventional stack.

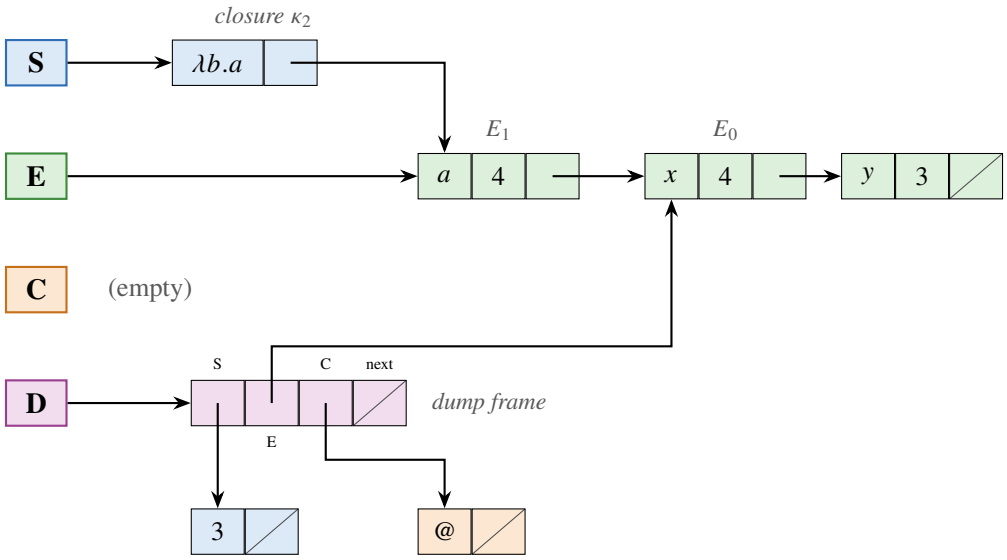


Figure 8: The machine's memory after step 7 of the $K\ x\ y$ trace. Cells with a dot hold a pointer, and a slash marks the end of a list. The stack holds one closure whose environment is E_1. The dump frame remembers the outer computation: stack $[3]$, environment E_0, and control $[@]$. The environments form linked lists with a shared tail.

- **Comparison with substitution.** The closure κ_2 in step 8 is the machine's representation of $\lambda b.4$, the intermediate term in (1). The two β-steps of (1) are the two Call transitions (steps 6 and 9). Everything else is bookkeeping that substitution hides.

Figure 8 shows the machine's memory just after step 7. Nothing in the machine is ever overwritten: E_1 extends E_0 by pointing at it, and the dump frame points at the same E_0 that the closure's environment chain ends in.

5.3 Lexical scope in action

Finally, the term T from (2). Its trace takes 16 steps, and we summarise the important moments rather than listing all of them:

1. The argument $\lambda z.x$ is evaluated first, by Abs in E_0, giving $\kappa = \langle \lambda z.x,\ E_0 \rangle$.
2. The outer call binds $f \mapsto \kappa$, producing $E_f = [f \mapsto \kappa] \cdot E_0$.
3. The inner call $(\lambda x.f\ x)\ y$ binds $x \mapsto 3$, producing $E_x = [x \mapsto 3] \cdot E_f$.
4. Evaluating $f\ x$ in E_x pushes 3 and κ, and Call runs the body x of κ in $[z \mapsto 3] \cdot E_0$, *not* in E_x.
5. Var finds $x \mapsto 4$. The result is 4, as substitution predicted.

Figure 1 shows this picture. The dump reaches depth 3 at step 4: the outer call, the inner call and the call of κ are all pending at once.

6 What the machine computes

6.1 Values and read-back

The machine's results are machine values. For a constant, that is all there is to say. A closure, however, is not a λ-term, so to compare the machine with the λ-calculus we need to turn closures back into terms. Plotkin called this function *Real* [13]:

$$Real(c) = c \quad \text{for constants,} \qquad Real(\langle \lambda x.B,\ E \rangle) = (\lambda x.B)\big[Real(E(y_1))/y_1, \ldots, Real(E(y_k))/y_k\big],$$

where $y_1, \ldots, y_k$ are the free variables of $\lambda x.B$. In words: take the code and substitute in whatever its environment says about its free variables. For instance, $Real(\kappa_2) = Real(\langle \lambda b.a,\ [a \mapsto 4] \cdot E_0 \rangle) = \lambda b.4$.

Theorem 1 (Correctness, informally)**.** *Let M be a term whose free variables are bound to constants by E_0. Starting from $([\,], E_0, [M], [\,])$, the machine halts with value v on top of the stack if and only if M reduces under call-by-value to a value V, and in that case $Real(v) = V$. If M has no call-by-value value, the machine runs forever or gets stuck.*

Plotkin proved a theorem of this form for Landin's machine in 1975, in the same paper that defined the call-by-value λ-calculus [13]. To see why such a theorem holds, read each machine state back as a λ-term, using *Real* on the values and plugging the result into the computations saved on the dump. Then check that every transition either leaves that term unchanged (bookkeeping) or performs one β_v-step.

6.2 Consequences

- **The machine stops at WHNF.** If M evaluates to a function, the result is a closure whose body has *not* been evaluated. In section 7 this means that the closure returned by "factorial of 3" does not yet contain the number 6. It is a recipe that will produce 6 when used.
- **Evaluation order is fixed but rarely visible.** Arguments are evaluated right to left. For terms without side effects, the order only matters when two subterms fail differently: if both M and N diverge or get stuck, it determines which failure you see first.
- **Stuck states are possible.** The pure rules have no answer for an unbound variable, or for @ with a constant such as 4 in function position. The C program aborts in the first case, and in the second it silently consumes the @ and leaves both values on the stack (exercise 6).

7 Programming the machine

With only λ, variables and application, data has to be encoded as functions. This section uses the standard Church encodings and runs into two interesting obstacles, both caused by call-by-value.

7.1 Booleans and numbers

$$\begin{aligned} \mathit{true} &= \lambda x.\lambda y.x & \mathit{false} &= \lambda x.\lambda y.y \\ \underline{0} &= \lambda f.\lambda x.x & \underline{1} &= \lambda f.\lambda x.f\,x \\ \mathit{succ} &= \lambda n.\lambda f.\lambda x.\ f\ (n\ f\ x) & \mathit{times} &= \lambda m.\lambda n.\lambda f.\ m\ (n\ f) \\ \mathit{iszero} &= \lambda n.\ n\ (\lambda x.\mathit{false})\ \mathit{true} & \mathit{pred} &= \lambda n.\lambda f.\lambda x.\ n\ (\lambda g.\lambda h.\ h\ (g\ f))\ (\lambda u.x)\ (\lambda u.u) \end{aligned}$$

The numeral $\underline{n}$ applies its first argument n times. A boolean selects one of its two arguments, so "if b then P else Q" is written $b\,P\,Q$. The predecessor function is the well-known tricky one: it rebuilds the numeral one step behind, using the pair-like functions $\lambda g.\lambda h.h\,(g\,f)$.

The companion program checks results by asking yes/no questions, since booleans are easy to recognise on the stack: the machine value of *true* prints as the closure $\langle \lambda y.x,\ \ldots \rangle$ and *false* as $\langle \lambda y.y,\ \ldots \rangle$.

7.2 Obstacle 1: a strict *if*

Under call-by-value, $b\,P\,Q$ evaluates *both* P and Q before b gets to choose. That is wasteful at best. When one branch is a recursive call, it never terminates, because the recursion is unfolded whether or not it is needed.

The standard fix is to *delay* each branch by wrapping it in a λ, which the machine will not evaluate inside (rule Abs). Then the selected branch is called with a dummy argument:

$$\text{if } b \text{ then } P \text{ else } Q \quad \rightsquigarrow \quad b\ (\lambda d.P)\ (\lambda d.Q)\ \mathit{dummy}.$$

A function used this way, purely to postpone evaluation, is called a *thunk*.

n	transitions	max. dump depth
0	51	3
1	151	8
2	341	13
3	626	18
4	1026	23

Table 1: Cost of evaluating $\mathit{fac}\,\underline{n}$ to weak head normal form. The dump depth grows by exactly 5 per level of recursion. The number of transitions grows faster than linearly (exercise 8).

7.3 Obstacle 2: recursion without Y

The usual fixed-point combinator $Y = \lambda f.(\lambda x.f\ (x\,x))\ (\lambda x.f\ (x\,x))$ is useless under call-by-value. To evaluate $Y\,H$ the machine must evaluate the argument $x\,x$ before calling f, and $x\,x$ unfolds into $H\ (x\,x)$, whose argument must again be evaluated first, and so on forever. The fix is the same trick as before, delaying with a λ. Eta-expanding $x\,x$ to $\lambda y.x\,x\,y$ gives the call-by-value fixed-point combinator

$$Z = \lambda f.\ (\lambda x.\ f\ (\lambda y.\,x\,x\,y))\ (\lambda x.\ f\ (\lambda y.\,x\,x\,y)).$$

Now the argument passed to f is a λ, which is already a value, and the self-application happens only when the recursive function is actually called.

7.4 Factorial

Putting both fixes together:

$$\begin{aligned} H &= \lambda f.\lambda n.\ \mathit{iszero}\,n\ (\lambda d.\underline{1})\ (\lambda d.\ \mathit{times}\,n\,(f\ (\mathit{pred}\,n)))\ n \\ \mathit{fac} &= Z\,H \end{aligned}$$

The trailing n is the dummy argument, and any value would do. The companion program verifies $\mathit{fac}\,\underline{3} = \underline{6}$ without ever printing a numeral: $\mathit{iszero}\ (\mathit{pred}^5\ (\mathit{fac}\,\underline{3}))$ evaluates to *false* and $\mathit{iszero}\ (\mathit{pred}^6\ (\mathit{fac}\,\underline{3}))$ to *true*. Both checks are needed, because $\mathit{pred}\,\underline{0} = \underline{0}$ in this encoding, so the second alone would also accept any number below 6. Section 8 shows a more direct way to read a numeral.

7.5 What it costs

Table 1 gives the number of transitions and the maximum dump depth for evaluating $\mathit{fac}\,\underline{n}$ (to WHNF) on the companion program, and fig. 9 shows the dump depth at every step of $\mathit{fac}\,\underline{2}$.

The dump grows because the frames below a recursive call are waiting for it to finish. Some frames, however, belong to calls whose caller has nothing left to do afterwards. The identity example in section 5 saves such a frame, with an empty control. Return into such a frame just returns again. A machine that recognises these *tail calls* and does not push a frame for them uses less space (section 10.4 and exercise 11).

8 Primitives and constants

Real programs need numbers and arithmetic. These are added as *primitive* values, together with a function δ that says what each primitive does when applied. This is rule Prim:

$$(f : v : S,\ E,\ @ : C,\ D) \longrightarrow (\delta(f, v) : S,\ E,\ C,\ D).$$

No dump frame is needed: the primitive's work is done in a single step.

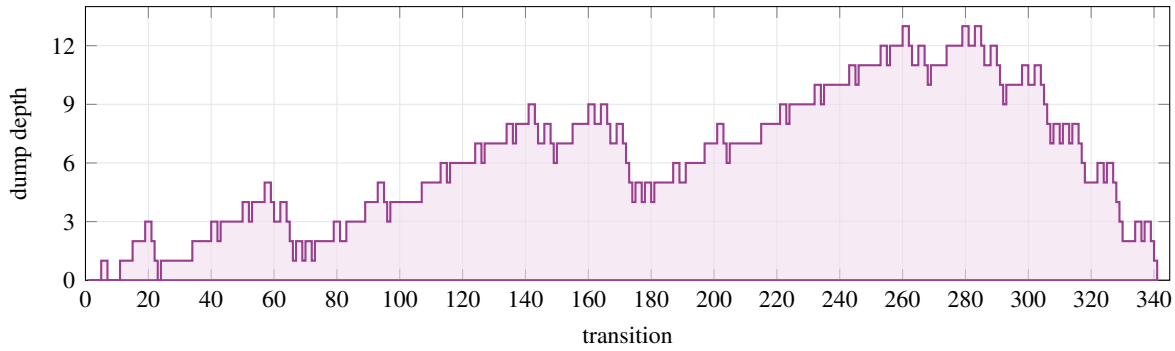


Figure 9: Dump depth during the 341 transitions of *fac* 2. Each rise is a call and each fall a return. The depth climbs in two stages, one per level of recursion: it peaks at 9 around transition 140, falls back only partway, then climbs to 13 between transitions 260 and 285 before the whole computation unwinds. Frames from the outer level are still pending while the inner level runs.

One argument. $\delta(SUCC, n) = n + 1$. With *SUCC* available, a Church numeral can be converted to a machine integer by applying it: 6 *SUCC* 0 evaluates to the constant 6. This is the easiest way to see the result of *fac* 3 (exercise 7).

Curried primitives. A two-argument operation such as addition receives its arguments one at a time. One solution is to let δ return a new primitive that remembers the first argument: $\delta(PLUS, m) = PLUS_m$ and $\delta(PLUS_m, n) = m + n$. This is a tiny closure in disguise.

Conditionals. A primitive receives its arguments already evaluated, so an *IF* primitive would suffer from exactly the strictness problem described above. Implementations therefore treat conditionals specially. The compiled machine in section 10 uses dedicated instructions.

The companion C program reserves a value tag `PRIM` and an enumeration of primitive operations (`PLUS`, `MINUS`, `IF`), but the PRIM case of its main loop is still an empty stub.

9 Inside an implementation

This section reads the companion program `secd23.c`, about 400 lines of C. It is deliberately small, and it is a good model of how the rules become code, including one way to get it wrong.

9.1 Representation

Expressions, values and environment bindings are each a C struct. Each struct has a `next` pointer, which lets it be threaded directly onto a list:

```
typedef enum {ID, LAM, APP, AT} exptag;           /* AT is the @ marker */
typedef enum {INT, PRIM, CLOSURE} whnftag;

typedef struct exp_cell  { struct exp_cell  *next; exptag tag;  ... } *EXP;
typedef struct whnf_cell { struct whnf_cell *next; whnftag tag; ... } *WHNF;
typedef struct env_cell  { struct env_cell  *next; char name; WHNF value; } *ENV;

typedef struct state_cell {
  WHNF stack;                 /* S: list of values, linked by next   */
  ENV environment;            /* E: list of bindings, linked by next */
  EXP control;                /* C: list of expressions and @ marks  */
  struct state_cell *dump;    /* D: list of saved states             */
} *STATE;
```

The stack is a list of `WHNF` cells and the control is a list of `EXP` cells, both linked through the cells' own `next` fields rather than through separate list nodes. Lists linked this way are called *intrusive*. A dump frame is just another `STATE`, so the dump is a list of states. Variable names are single characters.

9.2 From rules to code

The main loop of fig. 7 is a `while` over `machine->control || machine->dump`. The correspondence is direct:

rule	code
VAR	`push(valueof(x->expkind.id, machine->environment), machine)`. `valueof` searches the environment list and aborts when the name is unbound.
ABS	`push(mkclosure(body, bv, machine->environment), machine)`.
APP	Build the control cells N, M, @ and link them in front of the rest of the control (excerpt below).
CALL	Allocate a `STATE` holding the stack below the argument, plus the current environment and control, and push it on the dump. Bind the parameter in front of the closure's environment and start the body.
RETURN	Link the top of the stack in front of the saved stack, then restore the environment, control and dump from the frame.

Here is the APP case, with its debugging output removed:

```
} else if (x->tag == APP) {                    /* case 1c */
  F = copyexp(x->expkind.app.rator);
  A = copyexp(x->expkind.app.rand);
  e = mkat();                                  /* the @ marker */
  e->next = machine->control;
  F->next = e;
  A->next = F;
  machine->control = A;                        /* C := A : F : @ : C */
}
```

The calls to `copyexp` are the important detail. The next section explains why they are there.

9.3 A lesson in sharing

The original version of the program, `secd.c`, had the same code without the copies: it set `F->next` and `A->next` on the subterms of the program itself. The program built its terms in C by reusing named pieces such as *succ* in several places, so the term was not a tree but a DAG with shared nodes. That works for read-only data, but the `next` field is not read-only.

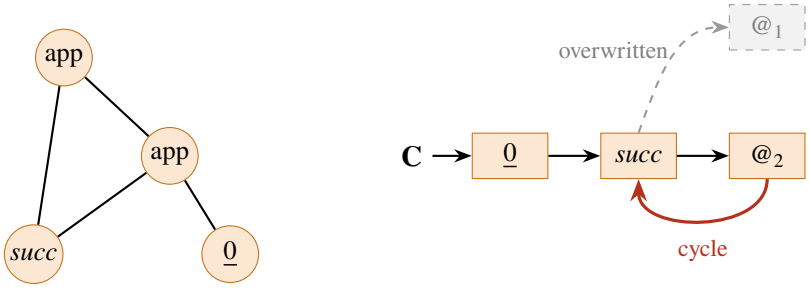
Consider *succ* (*succ* $\underline{0}$), where both occurrences of *succ* are the *same* node. The first APP sets *succ*.`next` = $@_1$. The second sets *succ*.`next` = $@_2$, overwriting the first, and $@_2$.`next` is the old rest of the control, which begins with *succ* itself. The control list now contains a cycle (fig. 10), and the machine applies *succ* over and over until it crashes. In the original program this happened within a few steps.

The fix is to never write `next` on a cell that might be shared. The program copies an expression cell (a shallow copy: only the cell, not its subterms) before linking it into the control, and copies a value cell before pushing it. Values stored in environments are copies too. Otherwise a value that sat on the stack would drag the rest of the stack along with it through its `next` field.

The lesson goes beyond this program. The abstract machine is purely functional: no rule ever modifies an existing structure. An implementation that saves memory by updating cells in place must make sure that no other part of the state can see those cells.

9.4 Memory

The program never frees anything. A real implementation needs a garbage collector, and the $K\ x\ y$ example shows why a stack discipline is not enough: the environment E_1 outlives the call that created it, because



(a) one shared *succ* node

(b) control after two App steps

Figure 10: How sharing broke the original program. (a) The term *succ* (*succ* $\underline{0}$) built with a single *succ* node. (b) Each App step writes the `next` field of the node it puts on the control. The second write overwrites the first, and the control list loops back on itself.

κ_2 still points to it. Environments therefore live in the heap and are shared, and the structure of linked environments is sometimes called a *spaghetti stack*. SECD machines have always assumed a Lisp-style heap with garbage collection.

10 Compiling for the SECD machine

Look again at rule App. Each time an application reaches the front of the control, the machine re-derives the same postfix sequence $N, M, @$. It does this every time the code runs. Inside a loop or a recursive function, the same term is taken apart again and again. Since the sequence depends only on the term, not on any run-time value, it can be computed once, in advance. That is compilation.

10.1 Instructions

The compiled SECD machine, in the style of Henderson [9], replaces the control's λ-terms by *instructions*. Variables are replaced by *de Bruijn indices* [5]: the variable's position in the environment, counting from the front. This works because the shape of the environment at each point in the program is known statically. It is the list of enclosing binders. Lookup by name becomes lookup by position.

instruction	transition	
LD i	$(S,\ E,\ \mathsf{LD}\,i : C,\ D)$	$\to (E(i) : S,\ E,\ C,\ D)$
LDC k	$(S,\ E,\ \mathsf{LDC}\,k : C,\ D)$	$\to (k : S,\ E,\ C,\ D)$
LDF c	$(S,\ E,\ \mathsf{LDF}\,c : C,\ D)$	$\to (\langle c,\ E\rangle : S,\ E,\ C,\ D)$
AP	$(\langle c,\ E'\rangle : v : S,\ E,\ \mathsf{AP} : C,\ D)$	$\to ([\,],\ v : E',\ c,\ (S, E, C) : D)$
RTN	$(v : S,\ E,\ \mathsf{RTN} : C,\ (S', E', C') : D)$	$\to (v : S',\ E',\ C',\ D)$

A closure now pairs *code* c, an instruction list, with an environment. AP is the old Call, and RTN is the old Return, now triggered by an explicit instruction instead of an empty control.

10.2 The compiler

The compiler $C[[M]]\rho$ takes a term and a *compile-time environment* ρ, the list of variable names that the run-time environment will hold. It is three lines long, one per syntactic form:

$$
\begin{aligned}
C[[x]]\rho &= [\mathsf{LD}\ i] \quad \text{where } i \text{ is the position of } x \text{ in } \rho \\
C[[\lambda x.M]]\rho &= [\mathsf{LDF}\ (C[[M]](x : \rho) +\!\!+ [\mathsf{RTN}])] \\
C[[M\ N]]\rho &= C[[N]]\rho +\!\!+ C[[M]]\rho +\!\!+ [\mathsf{AP}]
\end{aligned}
$$

The third line is exactly rule App, applied once, at compile time, to every application in the program.

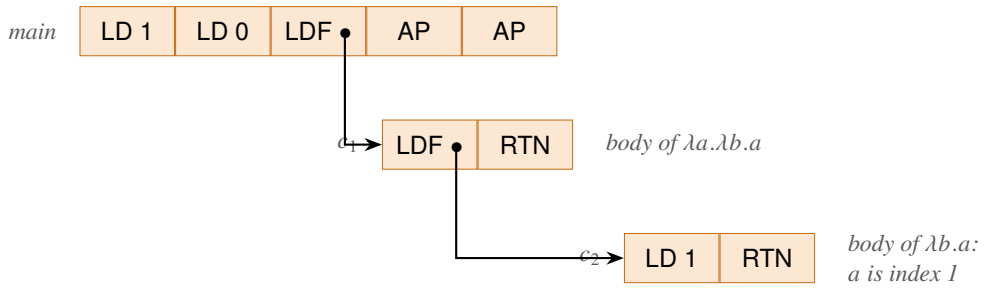


Figure 11: Compiled code for $K\ x\ y$. Each λ becomes a separate instruction list, referenced by the LDF that creates its closures. The code is never modified at run time, so it can be shared freely.

For $K\ x\ y$ with $\rho_0 = [x, y]$, so x is index 0 and y index 1:

$$C[[K\ x\ y]]\rho_0 = [\mathsf{LD}\ 1,\ \mathsf{LD}\ 0,\ \mathsf{LDF}\ c_1,\ \mathsf{AP},\ \mathsf{AP}] \quad \text{where} \quad c_1 = [\mathsf{LDF}\ c_2,\ \mathsf{RTN}], \quad c_2 = [\mathsf{LD}\ 1,\ \mathsf{RTN}].$$

In c_2, the body of $\lambda b.a$, the compile-time environment is $[b, a, x, y]$, so a is index 1. Figure 11 draws the code, which is a set of instruction lists linked by the pointers inside LDF.

Running it with run-time environment $[4, 3]$:

#	instr.	S	E	C	D
0	start	[ ]	[4, 3]	[LD 1, LD 0, LDF c_1, AP, AP]	[ ]
1	LD 1	[3]	[4, 3]	[LD 0, LDF c_1, AP, AP]	[ ]
2	LD 0	[4, 3]	[4, 3]	[LDF c_1, AP, AP]	[ ]
3	LDF	[$\langle c_1,\ [4, 3]\rangle$, 4, 3]	[4, 3]	[AP, AP]	[ ]
4	AP	[ ]	[4, 4, 3]	c_1	[([3], [4, 3], [AP])]
5	LDF	[$\langle c_2,\ [4, 4, 3]\rangle$]	[4, 4, 3]	[RTN]	[([3], [4, 3], [AP])]
6	RTN	[$\langle c_2,\ [4, 4, 3]\rangle$, 3]	[4, 3]	[AP]	[ ]
7	AP	[ ]	[3, 4, 4, 3]	c_2	[([ ], [4, 3], [ ])]
8	LD 1	[4]	[3, 4, 4, 3]	[RTN]	[([ ], [4, 3], [ ])]
9	RTN	[4]	[4, 3]	[ ]	[ ]

Compare this with the interpreted trace in section 5: the same calls and returns happen at the same moments, but the two App steps are gone (9 transitions instead of 11), and every variable lookup is an index rather than a search by name. Notice also that the sharing bug of section 9.3 cannot arise here. The control is a pointer into immutable code, and advancing it never writes anything.

10.3 Conditionals

With primitive booleans, a conditional compiles to two code blocks and a selection instruction. The dump saves the place to continue at afterwards:

$$\begin{aligned} &\mathsf{SEL} \quad (b : S,\ E,\ \mathsf{SEL}\ c_t\ c_f : C,\ D) \to (S,\ E,\ c_b,\ C : D) \\ &\mathsf{JOIN} \quad\ \ (S,\ E,\ [\mathsf{JOIN}],\ C : D) \to (S,\ E,\ C,\ D) \end{aligned}$$

where c_b is c_t or c_f depending on b, and both branches end in JOIN. Only the selected branch is ever executed, so the thunks of section 7 are no longer needed.

10.4 Tail calls

An application whose result is immediately returned, compiled as $\ldots, \mathsf{AP}, \mathsf{RTN}$, pushes a dump frame only for RTN to pop it straight away. A *tail-call* instruction avoids this by not saving a frame at all:

$$\mathsf{TAP}: \quad (\langle c, E'\rangle : v : S,\ E,\ [\mathsf{TAP}],\ D) \ \to\ ([\,],\ v : E',\ c,\ D).$$

When the body of c finishes, it returns directly to the caller's caller. With TAP, a loop written as tail recursion runs in constant dump space. Scheme makes this behaviour mandatory, and it is the reason the Scheme standard talks about "proper tail recursion".

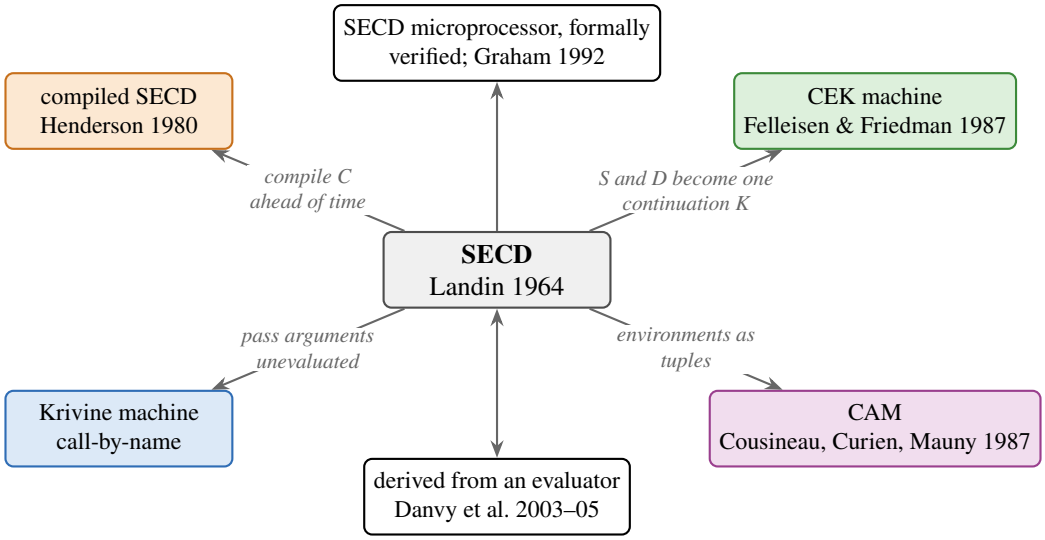


Figure 12: The SECD machine and some of the machines and results built on it.

11 Relatives and descendants

The SECD machine is the ancestor of a family of abstract machines. Each one changes one of its design decisions (fig. 12).

Compiled SECD.
Henderson's book [9] presents the instruction set of section 10, extended with lists, primitives and conditionals, and uses it to implement *Lispkit Lisp*, a small pure Lisp whose compiler is written in itself.

The CEK machine.
Felleisen and Friedman [6] observed that S and D together just describe "what to do next", and replaced them by a single *continuation* K. The result has three registers, Control, Environment and Kontinuation, and has become the standard machine for explaining control operators such as `call/cc`.

The Krivine machine.
A machine for *call-by-name*: an argument is not evaluated but pushed as a closure (a thunk), and evaluated only if the function uses it [10]. It needs no dump at all, and is remarkably small. Adding sharing of evaluated thunks leads to lazy machines such as those used for Haskell [12].

The Categorical Abstract Machine.
Cousineau, Curien and Mauny derived a machine from categorical combinators in which environments are nested tuples and variables become projections [3]. It was the basis of early implementations of the Caml language, whose name comes from it.

Hardware.
The SECD design has also been built as a microprocessor, whose correctness was formally verified in the HOL theorem prover [8].

Derivation, not invention.
Danvy and colleagues showed that the SECD machine can be *derived* mechanically from an ordinary recursive evaluator, through closure conversion, transformation into continuation-passing style, and defunctionalisation of the continuations [1, 4]. On this reading the dump is simply the defunctionalised continuation of the evaluator, which makes the intuition of section 3 precise.

12 Summary

The SECD machine evaluates call-by-value λ-calculus with four lists. It replaces substitution by *environments*, represents function values as *closures* so that scoping stays lexical, schedules work in *postfix* order on the control, and keeps pending calls on the *dump*, an explicit call stack. Each transition does a small, fixed amount of work. The machine computes exactly the call-by-value values of λ-terms, read back from closures. Pre-computing the control turns it into a compiled virtual machine, and its ideas (closures, heap-allocated environments, an explicit continuation) appear in essentially every implementation of a functional language.

Exercises

Exercise 1. Trace the machine on $(\lambda x.\, x\, x)\ (\lambda y.y)$ starting from $E_0 = [\,]$. What is the result, and what is the maximum depth of the dump?

Exercise 2. Trace $(\lambda f.\, f\ y)\ (\lambda z.z)$ in $E_0 = [x \mapsto 4, y \mapsto 3]$. At the moment the body of $\lambda z.z$ is entered, the dump holds two frames. Explain what each frame is waiting for. *(Answer: the result is* 3*.)*

Exercise 3. Let $\Omega = (\lambda x.x\, x)\ (\lambda x.x\, x)$. Show that the machine runs forever on Ω. Does the dump grow without bound? How does the answer change on a compiled machine that uses TAP?

Exercise 4. Change App to evaluate the function before the argument. Which other rule must change, and how? Find a term (it may get stuck or diverge) whose observable behaviour differs between the two orders.

Exercise 5. Modify Call so that the body runs in the *caller's* environment extended with the parameter, ignoring the closure's environment. Run T from (2) again and confirm that the result becomes 3. Which other parts of the machine become unnecessary?

Exercise 6. In `secd23.c`, applying a constant (for example the term $x\ y$ in E_0) does not stop the machine. Find out what happens to the stack, and make the program report the error instead.

Exercise 7. Implement rule Prim in `secd23.c` with a single primitive *SUCC*, bound in the initial environment to a single-letter name. Use it to print the value of $fac\ \underline{3}$ as the integer 6. How many transitions does the conversion add?

Exercise 8. Table 1 shows the number of transitions growing faster than linearly in n, even though $fac\ \underline{n}$ stops at WHNF and never multiplies out its result. Explain where the work goes. *Hint:* how much work does $iszero\ (pred^k\ \underline{n})$ take, and how many of these does the recursion evaluate?

Exercise 9. Implement *Real* for the C program: print a closure as the λ-term it represents. Check that κ_2 from the $K\ x\ y$ trace prints as $\lambda b.4$.

Exercise 10. Write the compiler C of section 10 and the compiled machine in a language of your choice. Compile *fac* and compare the number of transitions with table 1.

Exercise 11. Add TAP to your compiled machine, and make the compiler emit it for applications in tail position. Measure the maximum dump depth of $fac\ \underline{n}$ again.

Exercise 12. Why does $Y\ H$ diverge on the SECD machine while $Z\ H$ does not? Trace the first few steps of each until the difference is clear.

A Reference card

Interpreted machine	(start $([\,], E_0, [M], [\,])$; halt on $(v : S, E, [\,], [\,])$)	
VAR	$(S, E, x : C, D)$	$\to (E(x) : S, E, C, D)$
ABS	$(S, E, \lambda x.B : C, D)$	$\to (\langle \lambda x.B, E \rangle : S, E, C, D)$
APP	$(S, E, M\,N : C, D)$	$\to (S, E, N : M : @ : C, D)$
CALL	$(\langle \lambda x.B, E' \rangle : v : S, E, @ : C, D)$	$\to ([\,], E'[x \mapsto v], [B], (S, E, C) : D)$
PRIM	$(f : v : S, E, @ : C, D)$	$\to (\delta(f, v) : S, E, C, D)$
RETURN	$(v : S, E, [\,], (S', E', C') : D)$	$\to (v : S', E', C', D)$
Compiled machine		
LD i	$(S, E, \mathsf{LD}\, i : C, D)$	$\to (E(i) : S, E, C, D)$
LDC k	$(S, E, \mathsf{LDC}\, k : C, D)$	$\to (k : S, E, C, D)$
LDF c	$(S, E, \mathsf{LDF}\, c : C, D)$	$\to (\langle c, E \rangle : S, E, C, D)$
AP	$(\langle c, E' \rangle : v : S, E, \mathsf{AP} : C, D)$	$\to ([\,], v : E', c, (S, E, C) : D)$
TAP	$(\langle c, E' \rangle : v : S, E, [\mathsf{TAP}], D)$	$\to ([\,], v : E', c, D)$
RTN	$(v : S, E, \mathsf{RTN} : C, (S', E', C') : D)$	$\to (v : S', E', C', D)$
SEL	$(b : S, E, \mathsf{SEL}\, c_t\, c_f : C, D)$	$\to (S, E, c_b, C : D)$
JOIN	$(S, E, [\mathsf{JOIN}], C : D)$	$\to (S, E, C, D)$
Compiler		
variable	$C[\![x]\!]\rho$	$= [\mathsf{LD}\ \mathit{index}(x, \rho)]$
abstraction	$C[\![\lambda x.M]\!]\rho$	$= [\mathsf{LDF}\ (C[\![M]\!](x : \rho) +\!\!+ [\mathsf{RTN}])]$
application	$C[\![M\,N]\!]\rho$	$= C[\![N]\!]\rho +\!\!+ C[\![M]\!]\rho +\!\!+ [\mathsf{AP}]$

References

[1] M. S. Ager, D. Biernacki, O. Danvy, and J. Midtgaard. A functional correspondence between evaluators and abstract machines. In *Proceedings of the 5th ACM SIGPLAN International Conference on Principles and Practice of Declarative Programming (PPDP)*, pages 8–19, 2003.

[2] H. P. Barendregt. *The Lambda Calculus: Its Syntax and Semantics*. North-Holland, revised edition, 1984.

[3] G. Cousineau, P.-L. Curien, and M. Mauny. The categorical abstract machine. *Science of Computer Programming*, 8(2):173–202, 1987.

[4] O. Danvy. A rational deconstruction of Landin's SECD machine. In *Implementation and Application of Functional Languages (IFL 2004)*, volume 3474 of *LNCS*, pages 52–71. Springer, 2005.

[5] N. G. de Bruijn. Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the Church–Rosser theorem. *Indagationes Mathematicae*, 34:381–392, 1972.

[6] M. Felleisen and D. P. Friedman. Control operators, the SECD-machine, and the λ-calculus. In *Formal Description of Programming Concepts III*, pages 193–217. North-Holland, 1987.

[7] A. J. Field and P. G. Harrison. *Functional Programming*. Addison-Wesley, 1988.

[8] B. T. Graham. *The SECD Microprocessor: A Verification Case Study*. Kluwer Academic Publishers, 1992.

[9] P. Henderson. *Functional Programming: Application and Implementation*. Prentice-Hall, 1980.

[10] J.-L. Krivine. A call-by-name lambda-calculus machine. *Higher-Order and Symbolic Computation*, 20(3):199–207, 2007.

[11] P. J. Landin. The mechanical evaluation of expressions. *The Computer Journal*, 6(4):308–320, 1964.

[12] S. L. Peyton Jones. *The Implementation of Functional Programming Languages*. Prentice-Hall, 1987.

[13] G. D. Plotkin. Call-by-name, call-by-value and the λ-calculus. *Theoretical Computer Science*, 1(2):125–159, 1975.